

Audion Get Tools

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

Version 2.12.0 · 2026-08-24 · 206.3 MB

- [Direct download](#) — unmetered, no rate limits
- [Project page](#) — every version and how to install

SHA-256: 3533c34fc091a8a9c6a81500c9f99cfa848f76b77792aecd79b3a74d9f319381

Portable CMD/PowerShell tooling for installing, updating, checking, exporting, and importing WinGet packages on Audion workstations.

Main entry point: `Launcher-Audion-Get.cmd`.

GUI entry point: `launcher_gui.cmd`.

Use `Launcher-Audion-Get-RU.cmd` for Russian UI text.

Current Model

- The old `common` / `extended` split is retired.
- Bulk install and update flows use thematic lists.
- Every list run is controlled with `Y/N/Q`; `Enter` means `Yes`.
- The GUI replaces console confirmation with explicit checkbox groups and runs only selected WinGet IDs.
- Installed package selectors use `Program name | ID` so uninstall remains readable and exact.

- GUI batch uninstall mirrors the thematic groups and adds `Custom` plus `Other installed`.
- Some uninstallers open UAC or their own confirmation UI; the GUI waits for the user's choice, verifies with `winget list --id <ID> -e`, and continues the batch.
- The GUI has a draggable splitter between the command pane and the live terminal; the terminal width is remembered.
- Launchers keep the terminal open with `pause`, then return to their own menu.
- Cancelling FZF with `Esc` returns to the menu without an error.
- Script folders live under `system_core\winget\`.
- The MSVC launcher only exposes the proven flows: install MSVC Legacy, install MSVC 2015+, update MSVC 2015+ x86/x64.

Structure

Audion-Get\

Launcher-Audion-Get.cmd

launcher_gui.cmd

Launcher-Audion-Get-RU.cmd

cli\Launcher-Audion-MSVC-Legacy.cmd

cli\Launcher-Audion-MSVC-Legacy-RU.cmd

cli\Launcher-Audion-Tools.cmd

cli\Launcher-Audion-Tools-RU.cmd

cli\Launcher-Audion-Check-Apps.cmd

cli\Launcher-Audion-Check-Apps-RU.cmd

cli\Launcher-Audion-Install-Apps.cmd

cli\Launcher-Audion-Install-Apps-RU.cmd

cli\Launcher-Audion-Update-Apps.cmd

cli\Launcher-Audion-Update-Apps-RU.cmd

config\

system.txt

dev.txt

ai.txt

pkms.txt

office.txt

media.txt

browsers-vpn.txt

hardware-benchmarks.txt

custom.txt

pins.txt

system_core\

fzf.exe

powershell\

winget\

scripts\

package_apps\

install_apps\

check_apps\

export_import\

msvc_legacy_updates\

install\

licenses\

logs\

release\

._runtime\

Thematic Lists

- `config\system.txt` — runtimes, terminals, archivers.
- `config\dev.txt` — developer tools.
- `config\ai.txt` — AI tools.
- `config\pkms.txt` — PKMS, notes, knowledge bases.
- `config\office.txt` — office, document, and reading tools.
- `config\media.txt` — images, audio, and video; blank lines split sections.
- `config\browsers-vpn.txt` — browsers, VPN, network and communication clients.
- `config\hardware-benchmarks.txt` — hardware diagnostics, benchmarks, utility tools.
- `config\custom.txt` — user-entered IDs installed through GUI "Install by ID" or added manually.
- `config\installed_uninstall_aliases.yaml` — installed-only ARP/MSIX/runtime ID patterns used to group uninstall and grouped update checkboxes.
- `config\pins.txt` — packages for `winget pin`.

The PKMS order is intentional: `Notion.Notion`, `Notion.NotionCalendar`, then Obsidian, Joplin, Evernote, and the remaining PKMS tools. Office/document tools such as LibreOffice, Acrobat Reader, and Calibre live in the separate Office group.

GUI

`launcher_gui.cmd` starts the NiceGUI/pywebview shell with checkbox-driven package operations, a live terminal, and a manual command bar.

- Install scans installed IDs and shows only missing project packages; checked IDs are installed without reinstalling what is already present.
- Update has four paths: preview the current `winget upgrade` list as `Name | ID | current -> available`, update everything returned by `winget upgrade`, update visible available updates by project group, or update checked items from the flat available-update list. Groups with no available updates show a clear no-updates message.
- Checkbox blocks include a name/ID filter and select/clear buttons. With a filter active, select/clear acts on the visible matches only.

- `Install / uninstall by ID` supports search, adding exact search-found IDs to a selected list/GUI group, typed-ID install/uninstall, and grouped batch uninstall.
- The System group also carries `.NET Framework 3.5`, which is a Windows optional feature rather than a WinGet package: Audion Get Tools enables it with DISM (payload comes from Windows Update) and hides it once it is enabled.
- The Portable window opens with `Google Chrome (web installer)`: it downloads the ~12 MB Google web installer for the current system language and architecture and installs it silently.
- Every package checkbox has small buttons inside its card: the green down arrow downloads the WinGet installer into `output\Downloads` without installing anything, and the blue arrow opens the page where builds can be picked by hand - GitHub `releases/latest`, TechPowerUp, or the vendor download page.
- The crimson archive button downloads the zip or standalone build - `winget download --installer-type zip|portable` - into `output\Portable\<product name>`. It is rendered only for the ids in `PACKAGE_ARCHIVE_TYPES`: every package was probed with `winget show --installer-type`, and types such as `inno (zip)` were left out because that is an installer inside an archive.
- A second table, `PACKAGE_GITHUB_BUILDS`, covers the case where the vendor ships a build the WinGet manifest does not carry, so `winget download` can never produce it. The value is a repository and two regular expressions, one for the installer and one for the archive (`Eugeniy/tabby` → `tabby-[\d.]+-setup-x64\.exe` and `tabby-[\d.]+-portable-x64\.zip`). Both buttons draw from one release, so they cannot hand out different versions - a release lands on GitHub before the manifest catches up. The second entry is `Zarestia-Dev/rc1one-manager`: the WinGet manifest carries the msi alone and trails the release, and the portable build never reaches it at all. This vendor also spells the architecture differently in the two names - `x64` in the installer, `x86_64` in the portable archive - so each button has its own expression rather than one shared pattern.
- Fallback for everything else: when `winget download` produces nothing and the package page points at GitHub, the file is looked up in that repository's latest release. `pick_windows_asset` chooses it - other systems and architectures, checksums, debug symbols and `.blockmap` files are refused, an explicit x64 build wins, and for archives so does the one that says `portable`. When nothing fits it returns empty, and the message carries both failures rather than a guess.
- The GitHub API allows 60 anonymous calls an hour and runs out quietly. So the release file list is read through the API first and, on any error from it, off the `releases/latest` and

`releases/expanded_assets/<tag>` pages, where there is no quota at all. The download link comes from the same answer, so nothing is asked of GitHub between finding the file and fetching it.

- Download layout: installers land flat in `output\Downloads` (including `ChromeSetup.exe`), archives and standalone builds in `output\Portable\<name>`, installation bundles in `output\Install\<name>`. The folder name comes from the checkbox caption and keeps the vendor's spelling (`MPC-BE`, `Notepad++`, `MSVC All-in-One (TechPowerUp)`); no WinGet id appears in a path. Clicking them never toggles the checkbox, and the page button keeps working while a batch is running.
- Why both: WinGet installs whatever the manifest ships, silently. Cross-platform and portable builds, installer options such as the PowerShell Explorer context-menu entries, and Microsoft Update self-update live on the vendor page. `Microsoft.PowerShell` is the clearest case - WinGet has an MSIX bundle only, so the MSI with those options can only come from GitHub.
- `Classic scripts` offers both all-in-one VC++ bundles - TechPowerUp and `abbodi1406/vcredist` - each with a download button and a blue arrow to its latest-release page. Both carry VC++ 2012, which has no WinGet package. Audion Get Tools downloads and unpacks them; running `install_all.bat` or `VisualCppRedist_AI0.exe /ai` stays a manual, elevated step.
- MSVC is two install groups: `MSVC runtime (2015+)`, the only one a modern system needs, and `Advanced: MSVC 2005-2013` for old software. Both hints point at the all-in-one bundles, which do the whole family in one pass and include 2012. Uninstall still lists every VC++ runtime in one block.
- `Service procedures` has `Check MSVC runtimes`: real versions from the registry, the 2015+ family compared with WinGet, every Programs-and-Features entry listed, and 2012 marked as covered only by the all-in-one bundles.
- `Classic scripts` can also install both bundles. Those actions ask for confirmation first, require Administrator rights, and write the runtime registry state before and after, because `install_all.bat` reports success whatever happens.
- The Portable window pairs `Google Chrome (web installer)` with a download-only icon on the same row.
- Parameter windows name their run button after the action - `INSTALL`, `UPDATE`, `UNINSTALL`, `SEARCH`, `PIN` - at a reserved width.

- **AI Package Planner** is an optional child section for LLM-assisted package suggestions. It manages provider keys/models/prompts locally, validates suggestions with WinGet search, writes a reviewable plan, and lets the user run only selected exact validated IDs through the normal Audion package path.
- In **AI Package Planner**, **AI task** is what to find or prepare, **Instruction template** is a saved AI behavior preset, and **AI planner instruction** is the rule set for how the AI should reason, validate WinGet IDs, and format the plan.
- Grouped uninstall organizes installed packages by the same thematic blocks as install/update. User-entered IDs are shown under **Custom**; installed IDs outside the project lists are shown under **Other installed**.
- Protected uninstall labels and confirmation guard App Installer, Terminal, PowerShell, MSVC/.NET/WindowsAppRuntime, VCLibs, WSL, Microsoft Edge, and similar shared dependencies.
- ARP/MSIX/runtime aliases can be mapped into uninstall and grouped update views without adding those IDs to install lists.
- Installed-package discovery loads asynchronously and uses a shared cache, so the GUI stays responsive while WinGet scans a large machine. The grouped uninstall view performs one serialized **winget list** scan and then filters that snapshot into thematic blocks.
- Unknown IDs installed through GUI **Install by ID** are appended to **config\custom.txt**.
- **Add ID to list** writes to the selected **config*.txt** list and, for GUI-backed groups, adds the same ID to **config\tool_manifest.yaml** so it appears as a normal checkbox.
- Batch uninstall does not stop on the first problematic ID; it continues through selected IDs and reports failures at the end.
- UAC/vendor GUI uninstallers are treated as user steps: the GUI logs **[WAIT]**, waits up to the configured timeout, and polls **winget list --id <ID> -e**.
- Maintenance actions live under the **Service procedures** root child section. It includes **Clear logs**, which cleans old files from **logs** while preserving the current operation log; **report** is left untouched.
- **Health / Doctor** checks WinGet availability, sources, installed/update counts, and the GUI doctor from inside the app.
- **Health / Doctor** also reports the WinGet MCP server (**winget mcp**, **WindowsPackageManagerMCPServer.exe**). Audion Get Tools does not run packages through it:

its two tools, `find-winget-packages` and `install-winget-package`, are a subset of the exact-ID paths and would lose live progress, logs, reports, and the protected-ID check.

- `Check installed IDs` is kept there as a diagnostic action; install/update flows already scan missing and updatable packages for normal use.
- Package batch actions write `action_summary` JSON/Markdown reports with selected packages, statuses, failures, skipped updates, and uninstall wait notes.
- The terminal panel preserves ANSI color, UTF-8/Cyrillic output, and can be resized with the splitter.
- WinGet runs under a Windows pseudo console (ConPTY), so the log shows live download bars and colors just like a normal PowerShell window. Set `AUDION_GET_CONSOLE_STREAM=0` to fall back to plain pipes.
- Download progress is one live line with a spinner that is refreshed in place, so a 300 MB download no longer fills the log with hundreds of rows. The file log keeps a coarse trail instead, every 1, 5, or 10 seconds depending on whether the download is above 10, 50, or 100 MiB.
- The update scan reads every installed package, including the ones whose installed version WinGet cannot read (most browsers), and the update table is parsed by column position so a localized Windows lists the same updates as an English one.
- Updating `Microsoft.AppInstaller` replaces `winget` itself: after that package the GUI waits for the execution alias to come back and falls back to the real package path under `Program Files\WindowsApps` instead of failing with `WinError 1920`.
- App Installer is therefore not a package checkbox. `Update WinGet` sits at the top of the update windows, runs alone, and warns that Audion Get Tools has to be restarted afterwards.
- Command windows group their actions in titled panels, and no field, filter, or select is left standing on the bare background.
- A command window never opens just to show more links: sections such as `Install / uninstall by ID` and `Import / export` lay their commands out in place, each as a panel with its own fields and a run button carrying the command name (`Search`, `Add ID to list`, `Install by ID`, `Uninstall by ID`). Only heavy checkbox windows still open separately. A field shown in several panels, such as the WinGet ID, keeps one value everywhere.
- Splitter state is stored in the pywebview/browser profile. For a release-default reset, remove `._runtime\webview` or run the release cleanup that removes `._runtime`; do not delete

`runtime\`, which contains the bundled Python payload. The release terminal default is `500px`; the built-in window default is `1600x900`.

Launchers

`Launcher-Audion-Get.cmd` is the main user entry point. It runs thematic install/update/pin flows and opens MSVC or Tools.

`cli\Launcher-Audion-Tools.cmd` is the service launcher for export/import, individual checks, point install/update from all thematic lists, package removal from lists, and WinGet search.

`cli\Launcher-Audion-MSVC-Legacy.cmd` is the MSVC launcher. Legacy 2005-2013 updates are intentionally disabled; only MSVC 2015+ x86/x64 is updated through `system_core\winget\install_apps\Update-Audion-MSVC-2015+.cmd`.

Each launcher also has a Russian `-RU.cmd` variant. `Exit` / `Q` closes the current launcher; finishing a child process returns to that launcher's menu.

List Run Policy

Bulk operations go through:

```
system_core\winget\scripts\Launch-WinGet-Lists.cmd  
  
system_core\winget\scripts\Run-WinGet-From-Lists.ps1
```

Confirmation mode:

```
Enter/Y = install or update  
  
N       = skip package  
  
Q       = quit current list
```

Logs are written to `logs\`; launcher menu temp files are written to `._runtime\`. Russian launchers use separate `_ru` temp files so they do not collide with English launchers.

Canonical Workbench labels

The top I/O panel uses the same vocabulary across Audion projects:

`Source`, `Add file...`, `Target`, `Reset`, `Delete`, `List`.

- `Source` and `Target` open the current routes.
- `Add file...` selects one file as the current Source without copying it.
- `Reset` drops unpinned history and restores project `input/output`; it does not delete files and keeps pinned paths.
- `Delete` clears the current Source and Target after one confirmation.
- `List` writes either the selected file or the current Source folder contents.

The address-row delete button clears that route. An external Source requires a separate confirmation; filesystem and project roots are protected.

Package Selection Safety

Search results are candidates, not approval to install. Review the exact WinGet ID, publisher, source, version, architecture, scope, and command action before execution. Prefer exact IDs over display-name guesses and keep thematic lists readable enough for manual review.

AI Package Planner may suggest candidates and group known Audion packages, but the final list remains an explicit operator decision. Planner output must pass exact-ID validation before install, update, or uninstall.

Repeatable Runs

Use the generated list or plan as the record of intent. Keep execution logs and reports for failures, skipped packages, source errors, and reboot requirements. A new search may return different versions or sources, so do not treat an old candidate list as a permanent lock file.